IAM — Console Dashboard Wise Important Interview Questions & Answers



---

1. What is IAM?
Answer:
IAM stands for Identity and Access Management.
It controls who can access AWS resources and what actions they can perform.
IAM mainly uses users, groups, roles, and policies to manage permissions.
It is one of the most important security services in AWS.
For example:
Suppose a developer needs permission to work with S3 but should not access RDS.
We can give the developer an IAM identity with only the required S3 permissions.
This follows the principle of giving only the permissions that are needed.
Example:
User
↓
IAM
↓
Policy
↓
S3
↓

Allow Access
Simple ga gurthupettuko:
IAM = Who can access what in AWS.

IAM Dashboard
2. What is the IAM Dashboard?
Answer:
The IAM Dashboard gives an overview of the security and identity configuration of an AWS account.
It shows important information such as account security status and IAM-related configuration.
We can use it to review security recommendations and access-related settings.
It is normally the starting point when managing IAM.
For example:
Suppose we open IAM after creating an AWS account.
The dashboard can help us check whether important security settings such as MFA and credentials are properly configured.
We can use this information to improve the account's security.
Example:
IAM Dashboard
↓
Security Information
↓
Users / Roles / Policies
↓
Review Access
Simple ga gurthupettuko:
IAM Dashboard = IAM security overview.

Users
3. What is an IAM User?
Answer:
An IAM User represents a person or application identity that needs access to AWS.
A user can have permissions through policies or groups.
Users can have credentials such as console passwords or access keys depending on the requirement.
For human users, AWS generally recommends using federation or IAM Identity Center where appropriate.
For example:
Suppose a company has a developer who needs to access AWS resources.
We can create an identity for that developer and give only the required permissions.
The developer can then access the allowed AWS services.
Example:
Developer
↓
IAM User
↓
Permissions
↓
AWS Services
Simple ga gurthupettuko:
User = Identity that needs AWS access.

---

4. What permissions can an IAM User have?
Answer:
An IAM User gets permissions through IAM policies.
Policies can be attached directly to the user or inherited through a group.
The policies define which AWS actions are allowed or denied.
The user can perform only the actions permitted by the effective policies.
For example:
Suppose a user needs to read files from S3.
We attach a policy allowing the required S3 read actions.
The user can then read the permitted objects but cannot automatically perform unrelated actions.
Example:
IAM User
↓
Policy
↓
s3:GetObject
↓
S3 Object
Simple ga gurthupettuko:
User Permissions = Policies decide allowed actions.

---

5. What is the Permissions tab of an IAM User?

Answer:
The Permissions section shows the policies that provide permissions to the user.
These permissions can come directly from policies attached to the user or through groups.
We can review what access the user has from this section.
It is useful when troubleshooting or auditing user permissions.
For example:
Suppose a developer says they cannot access an S3 bucket.
We can open the user's Permissions section and check which policies and groups are providing access.
This helps identify whether the required permission is missing.
Example:
IAM User
↓
Permissions
↓
Policies / Groups
↓
Allowed Actions
Simple ga gurthupettuko:
Permissions = Check what the user can do.

---

6. What is an IAM Group?
Answer:
An IAM Group is a collection of IAM Users.
We can attach policies to the group instead of attaching the same policy individually to every user.
Users in the group inherit the group's permissions.
Groups make permission management easier for teams with similar access requirements.
For example:
Suppose a company has 20 developers who all need the same S3 access.
Instead of attaching the policy to every developer individually, we create a Developers group and attach the policy to it.
All users in that group receive the required permissions.
Example:
Developers
↓
IAM Group
↓
S3 Policy
↓
S3 Access
Simple ga gurthupettuko:
Group = Manage common permissions for multiple users.

---

7. What is the difference between an IAM User and an IAM Group?
Answer:
An IAM User represents an individual identity, while an IAM Group is a collection of users.
Permissions can be assigned to users directly or inherited from groups.
Groups are useful when many users need the same permissions.
A group itself is not used to log in to AWS.
For example:
Suppose Ravi and Kumar are developers and both need the same S3 permissions.
We can put both users into a Developers group and attach one S3 policy to the group.
Both users then receive the group's permissions.
Example:
Ravi ─┐
├─ Developers Group
Kumar ─┘ ↓
S3 Policy
Simple ga gurthupettuko:
User = Individual, Group = Collection of Users.

IAM Roles
8. What is an IAM Role?
Answer:
An IAM Role is an identity that provides permissions to trusted entities.
Unlike a typical IAM User, a role does not have permanent username/password credentials.
AWS services, users, or federated identities can assume a role and receive temporary credentials.
Roles are heavily used for secure access between AWS services.
For example:
Suppose an EC2 instance needs to upload files to S3.
Instead of storing an AWS access key on the server, we attach an IAM Role to EC2.
EC2 can then obtain temporary credentials and access S3 according to the role's permissions.
Example:

EC2
↓
IAM Role
↓
S3 Policy
↓
S3 Bucket
Simple ga gurthupettuko:
Role = Temporary permissions for trusted identities/services.

---

9. Why do we use IAM Roles?
Answer:
IAM Roles allow applications and AWS services to access resources without storing long-term credentials.
They provide temporary credentials when the trusted entity assumes the role.
Roles are commonly used with EC2, Lambda, ECS, and other AWS services.
This is more secure and easier to manage than hardcoding access keys.
For example:
Suppose a Lambda function needs to read objects from S3.
We create a role with the required S3 permissions and assign the role to Lambda.
Lambda can then access S3 without storing an access key in the function code.
Example:
Lambda
↓
IAM Role
↓
S3 Permission
↓
S3
Simple ga gurthupettuko:
Role = Secure service-to-service access.

---

10. What is a Trust Relationship in an IAM Role?
Answer:
A Trust Relationship defines who or what is allowed to assume an IAM Role.
It is part of the role's trust policy.
The trusted principal could be an AWS service, AWS account, user, or federated identity depending on the configuration.
The trust policy answers the question: "Who can use this role?"
For example:
Suppose we create a role for EC2.
The trust policy can allow the EC2 service to assume that role.
EC2 can then obtain temporary credentials for the permissions attached to the role.
Example:
EC2
↓
Trust Policy
↓
Allowed to Assume Role
↓
IAM Role
Simple ga gurthupettuko:
Trust Policy = Who can assume the role?

---

11. What is the difference between Trust Policy and Permissions Policy?
Answer:
A Trust Policy defines who can assume an IAM Role.
A Permissions Policy defines what actions the role can perform after it is assumed.
Trust answers "Who can use this role?", while permissions answer "What can they do?"
Both are important for IAM Role security.
For example:
An EC2 service may be trusted to assume a role.
The permissions policy attached to that role may allow EC2 to read objects from S3.
So trust allows role assumption and permissions allow S3 access.
Example:
Trust Policy
↓
Who can assume?
↓
IAM Role
↓
Permissions Policy
↓

What can they do?
Simple ga gurthupettuko:
Trust = Who, Permissions = What.

Policies
12. What is an IAM Policy?
Answer:
An IAM Policy is a JSON document that defines permissions in AWS.
It specifies which actions are allowed or denied on which resources.
Policies can contain conditions that further restrict access.
IAM uses these policies to determine whether a request should be allowed.
For example:
Suppose an application only needs to read files from one S3 bucket.
We can create a policy that allows the required S3 read action only for that bucket.
The application cannot automatically access other AWS resources.
Example:
IAM Policy
↓
Allow
↓
S3 Read
↓
Specific Bucket
Simple ga gurthupettuko:
Policy = Permission rules written in JSON.

---

13. What are AWS Managed Policies?
Answer:
AWS Managed Policies are policies created and maintained by AWS.
They provide commonly required permissions for AWS services and job functions.
AWS can update these policies when service capabilities change.
They are convenient but may provide broader permissions than a custom policy.
For example:
Suppose a user needs common read-only permissions across AWS services.
We can attach an appropriate AWS-managed read-only policy instead of creating every permission manually.
This makes initial permission setup easier.
Example:
AWS
↓
Managed Policy
↓
IAM User / Role
↓
Permissions
Simple ga gurthupettuko:
AWS Managed Policy = Policy maintained by AWS.

---

14. What is a Customer Managed Policy?
Answer:
A Customer Managed Policy is an IAM policy created and managed by us.
We can define exactly which actions and resources the policy should allow or deny.
It provides more control than using a broad AWS-managed policy.
It is useful when an organization needs custom permissions.
For example:
Suppose developers should access only one specific S3 bucket.
We can create a customer-managed policy that allows only the required S3 actions on that bucket.
We can then attach the policy to the required users or roles.
Example:
Company
↓
Customer Managed Policy
↓
Specific S3 Permissions
↓
Developer Role
Simple ga gurthupettuko:
Customer Managed = We create and control the policy.

---

15. What is an Inline Policy?
Answer:
An Inline Policy is a policy embedded directly into a single IAM identity.

It has a one-to-one relationship with that user, group, or role.
It cannot be independently attached to multiple identities like a managed policy.
Inline policies can be useful when permissions must exist only for one specific identity.
For example:
Suppose one IAM Role needs a unique permission that should not be reused by other roles.
We can create an inline policy directly inside that role.
That policy belongs specifically to that role.
Example:
IAM Role
↓
Inline Policy
↓
Unique Permission
Simple ga gurthupettuko:
Inline Policy = Policy embedded inside one identity.

---

16. What is the difference between AWS Managed, Customer Managed, and Inline Policies?
Answer:
AWS Managed Policies are created and maintained by AWS.
Customer Managed Policies are created and maintained by our organization.
Inline Policies are embedded directly into one IAM identity.
The choice depends on how much control and reuse we need.
For example:
If many roles need the same custom permissions, we can create a customer-managed policy.
If AWS already provides suitable permissions, we may use an AWS-managed policy.
For a one-off identity-specific permission, an inline policy may be considered.
Example:
AWS Managed
↓
AWS maintains

Customer Managed
↓
We maintain

Inline
↓
Attached directly to one identity
Simple ga gurthupettuko:
Managed = Reusable, Inline = Identity-specific.

Policy Versions
17. What is a Policy Version?
Answer:
A policy version represents a particular version of an IAM managed policy.
When we update a managed policy, IAM can keep previous versions.
One version is marked as the default version and is used for authorization.
Policy versions are useful when managing and reviewing policy changes.
For example:
Suppose we have a policy that allows S3 read access.
Later we modify it to add another permission.
IAM can maintain the policy versions so we can review previous configurations.
Example:
Policy
↓
Version 1
↓
Version 2
↓
Version 3
↓
Default Version
Simple ga gurthupettuko:
Policy Version = Different versions of a managed policy.

MFA & Security Credentials
18. What is MFA in IAM?
Answer:
MFA stands for Multi-Factor Authentication.
It adds another authentication factor in addition to the normal sign-in credentials.

This helps protect AWS accounts and identities if a password is compromised.
MFA is especially important for privileged users and root access.
For example:
Suppose an administrator's password is stolen.
If MFA is enabled, the attacker also needs the additional authentication factor to sign in.
This provides an extra layer of account protection.
Example:
Password
+
MFA
↓
AWS Login
↓
Access
Simple ga gurthupettuko:
MFA = Password + Another authentication factor.

---

19. What are Access Keys?
Answer:
Access Keys are credentials used to make programmatic requests to AWS APIs.
They normally consist of an access key ID and a secret access key.
They should be protected carefully and should not be hardcoded into application source code.
For AWS workloads, IAM Roles are usually preferred over long-term access keys.
For example:
Suppose an application running on a developer laptop needs to call an AWS API.
It may use appropriate credentials configured through supported mechanisms.
For an EC2 application, an IAM Role is generally preferred instead of storing access keys.
Example:
Application
↓
AWS Credentials
↓
AWS API
↓
AWS Service
Simple ga gurthupettuko:
Access Key = Programmatic AWS authentication credential.

Account Settings
20. What is the IAM Password Policy?
Answer:
The IAM password policy controls requirements for IAM user console passwords.
It can define requirements such as minimum password length and character types.
It can also control password expiration and reuse settings where supported.
A strong password policy helps improve account security.
For example:
Suppose a company wants IAM user passwords to be strong.
The administrator can configure requirements such as minimum length and additional character types.
Users must then follow those requirements when creating or changing passwords.
Example:
IAM Password Policy
↓
Minimum Length
↓
Character Requirements
↓
Password Expiration
Simple ga gurthupettuko:
Password Policy = Rules for IAM user passwords.

---

21. What is an Account Alias?
Answer:
An AWS Account Alias provides a friendly name that can be used as part of the IAM sign-in URL.
It is easier to remember than the numeric AWS account ID.
It does not change the actual AWS account ID.
It mainly improves the usability of the sign-in address.
For example:
Instead of remembering a long account identifier, employees can use a company-specific alias in the IAM sign-in URL.
This makes the AWS console login address easier to recognize.
The underlying AWS account remains the same.
Example:

AWS Account
↓
Account Alias
↓
Friendly Sign-in URL
↓
IAM Login
Simple ga gurthupettuko:
Account Alias = Friendly name for AWS account sign-in.

Access Reports
22. What is an IAM Credential Report?
Answer:
A Credential Report provides account-level information about IAM users and their credentials.
It can show information related to passwords, access keys, MFA, and credential activity.
Security teams can use it to review IAM credential status.
It is useful for auditing and identifying credentials that may need attention.
For example:
Suppose a company has many IAM users.
The security team wants to check which users have MFA enabled and whether old credentials are still active.
A credential report can help them review this information.
Example:
IAM Users
↓
Credential Report
↓
Password / MFA / Access Keys
↓
Security Review
Simple ga gurthupettuko:
Credential Report = IAM credential audit report.

---

23. What is Access Advisor?
Answer:
IAM Access Advisor shows information about when IAM identities last accessed AWS services.
It can help identify permissions that may not be needed.
This supports the principle of least privilege by helping administrators review unused service permissions.
It is useful during IAM permission cleanup.
For example:
Suppose a developer role has permissions for ten AWS services but only uses S3 and EC2.
Access Advisor can help identify services that have not been accessed.
The administrator can review whether those unused permissions should be removed.
Example:
IAM Role
↓
Access Advisor
↓
Service Access History
↓
Unused Permissions Review
Simple ga gurthupettuko:
Access Advisor = See which AWS services were actually accessed.

Identity Providers
24. What is an IAM Identity Provider?
Answer:
An IAM Identity Provider allows AWS to trust identities managed by an external identity system.
It is commonly used for federation and single sign-on scenarios.
AWS IAM supports identity providers such as SAML 2.0 and OpenID Connect depending on the use case.
This reduces the need to create separate IAM users for every external identity.
For example:
Suppose a company already uses a corporate identity provider for employee authentication.
AWS can trust that identity provider and allow employees to access AWS using federation.
Employees do not need separate long-term IAM user passwords for every AWS account.
Example:
Corporate Identity Provider
↓
Federation
↓
AWS
↓

IAM Role

Simple ga gurthupettuko:

Identity Provider = Connect external identities with AWS.

Access Analyzer

25. What is IAM Access Analyzer?

Answer:

IAM Access Analyzer helps identify resources and permissions that may allow unintended access.
It can analyze resource policies and help find external access paths.
It can also help identify unused access depending on the analyzer features being used.
This is useful for improving security and least-privilege access.

For example:

Suppose an S3 bucket policy accidentally allows another AWS account to access the bucket.
Access Analyzer can help identify that external access path.
The administrator can then review and correct the policy if the access is not required.

Example:

AWS Resource
↓
Access Analyzer
↓
Policy Analysis
↓
External Access Finding
↓
Review / Fix

Simple ga gurthupettuko:

Access Analyzer = Find unintended access.

---

26. What is an Access Analyzer Finding?

Answer:

A Finding is a security result identified by IAM Access Analyzer.
It can indicate that a resource policy allows access from an external principal or another potentially unwanted access path.
Administrators can review the finding and decide whether the access is intentional.
If it is not required, the related policy can be modified.

For example:

Suppose an S3 bucket is unintentionally accessible by another AWS account.
Access Analyzer can generate a finding showing the external access.
The security team can investigate and remove the unnecessary permission.

Example:

Resource Policy
↓
Access Analyzer
↓
Finding
↓
External Access
↓
Review

Simple ga gurthupettuko:

Finding = Access Analyzer discovered something to review.

Important IAM Interview Questions

27. What is the Principle of Least Privilege?

Answer:

Least Privilege means giving an identity only the permissions required to perform its job.
We should avoid giving unnecessary permissions such as AdministratorAccess when they are not required.
This reduces the impact if credentials or an application are compromised.
It is one of the most important IAM security principles.

For example:

Suppose an application only needs to read files from one S3 bucket.
We should allow only the required S3 read action on that bucket instead of giving full S3 access.
This keeps the application's permissions as small as possible.

Example:

Application
↓
Only Required Permission
↓
Specific S3 Bucket

Simple ga gurthupettuko:

Least Privilege = Give only what is needed.

---

28. What is the difference between Authentication and Authorization?
Answer:
Authentication verifies who the user or identity is.
Authorization determines what that authenticated identity is allowed to do.
IAM is mainly responsible for managing authentication-related identities and authorization permissions in AWS.
Both are required for secure access.
For example:
When a user signs in successfully, AWS verifies the user's identity.
After login, IAM policies determine whether the user can access S3, EC2, or RDS.
So login is authentication and permission checking is authorization.
Example:
User
↓
Authentication
↓
Who are you?
↓
Authorization
↓
What can you do?
Simple ga gurthupettuko:
Authentication = Who are you?
Authorization = What can you do?

---

29. What is an IAM Policy Effect?
Answer:
The Effect element in an IAM policy specifies whether a permission is allowed or denied.
The two main values are Allow and Deny.
An explicit deny takes precedence over an allow.
This is an important concept when troubleshooting access problems.
For example:
Suppose a user has a policy allowing access to an S3 bucket.
If another applicable policy explicitly denies the same action, the explicit deny wins.
The user will not be able to perform that action.
Example:
Allow
+
Explicit Deny
↓
Access Denied
Simple ga gurthupettuko:
Explicit Deny = Wins over Allow.

---

30. What are Principal, Action, Resource, and Condition in an IAM Policy?
Answer:
Principal identifies who or what the policy applies to, mainly in resource-based policies and trust policies.
Action defines what AWS operation can be performed.
Resource identifies which AWS resource the action applies to.
Condition adds extra rules that must be satisfied.
For example:
Suppose a policy allows an application role to read objects from one S3 bucket only under specific conditions.
The policy can define the allowed action, target resource, trusted principal, and additional conditions.
Together these elements provide precise access control.
Example:
Principal
↓
Who?
↓
Action → What?
↓
Resource → Which?
↓
Condition → Under what rules?
Simple ga gurthupettuko:
Policy = Who + What + Which Resource + Conditions.